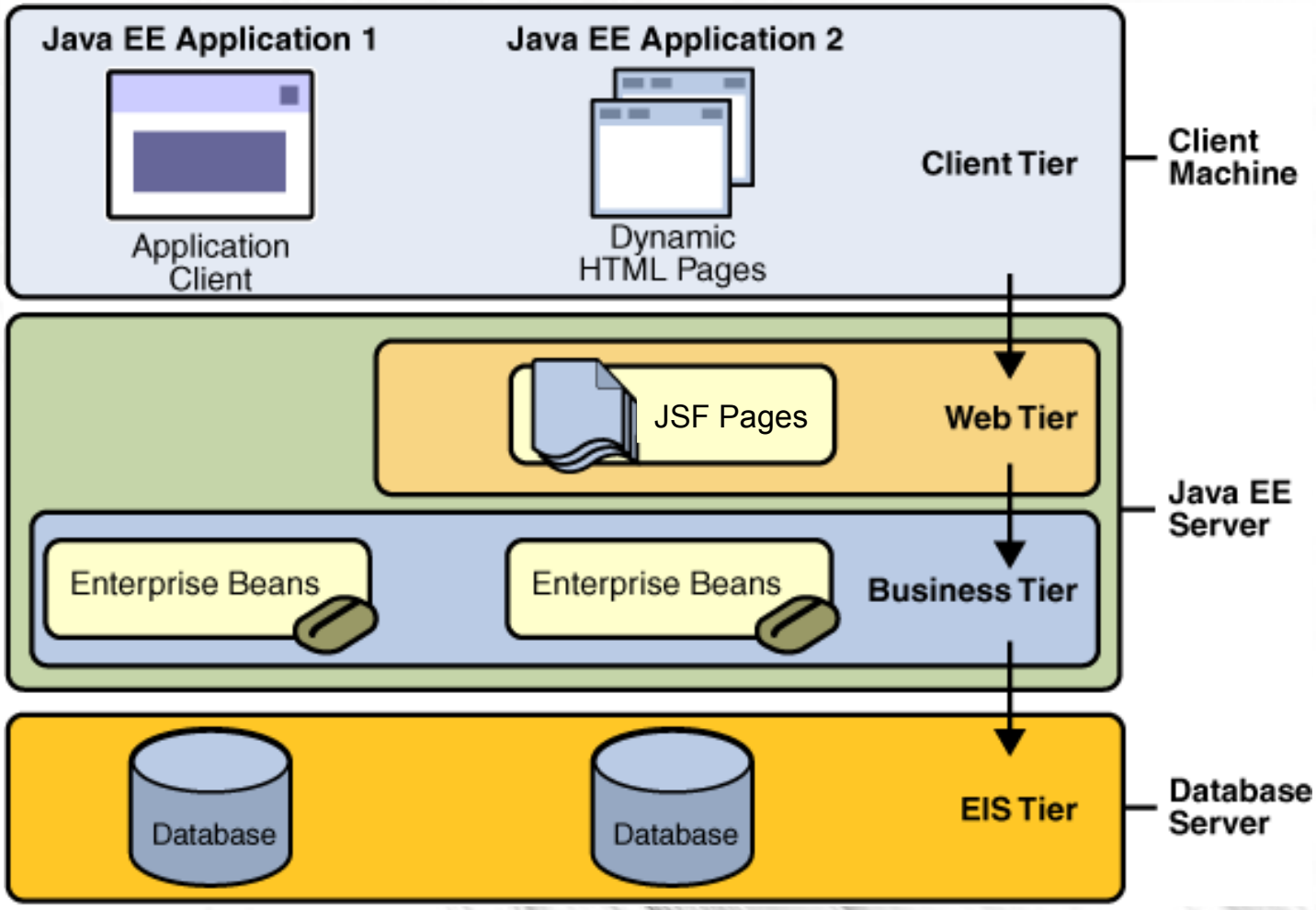
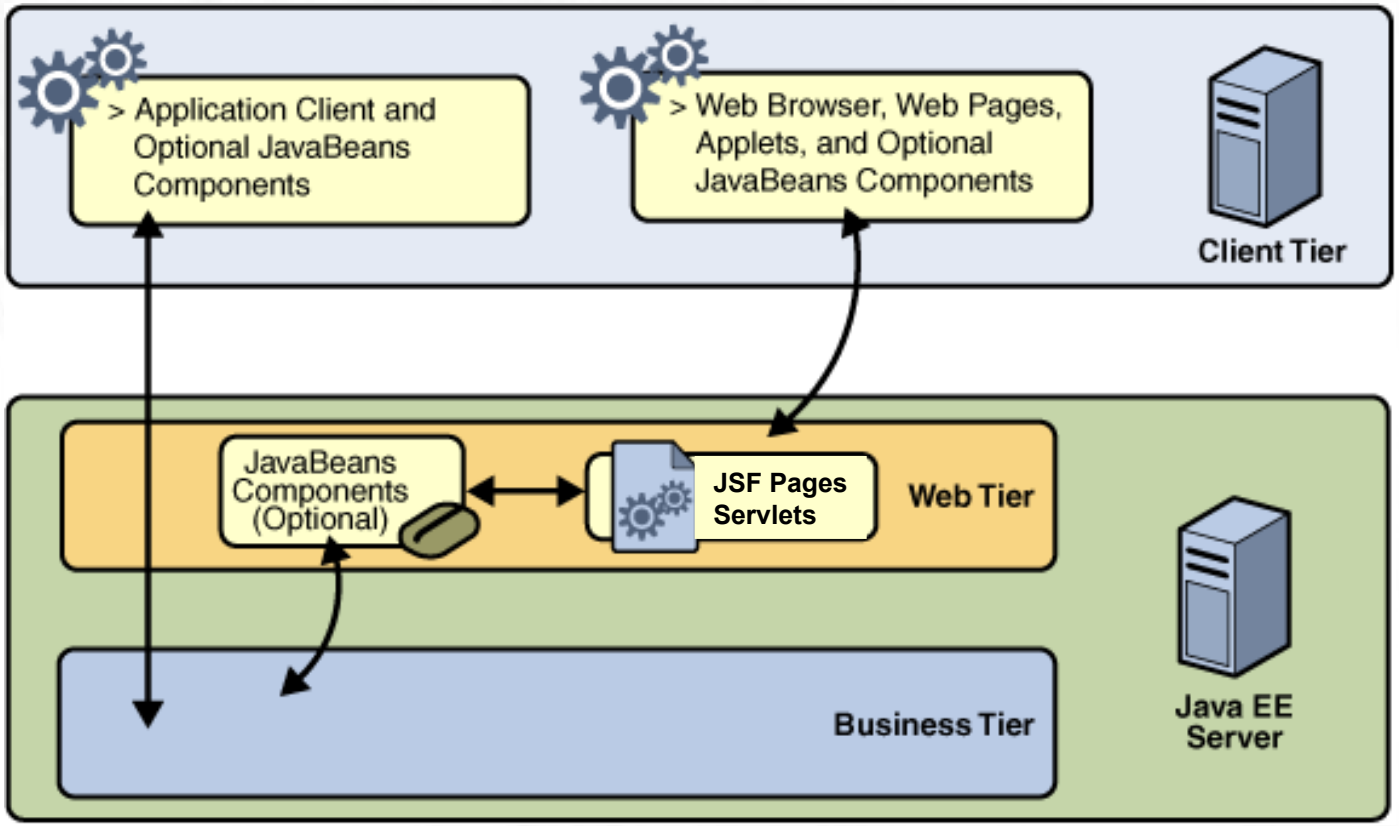
Java EE, EJB Design

Updated Chapters 20, 21from
EJB 3.0 Book

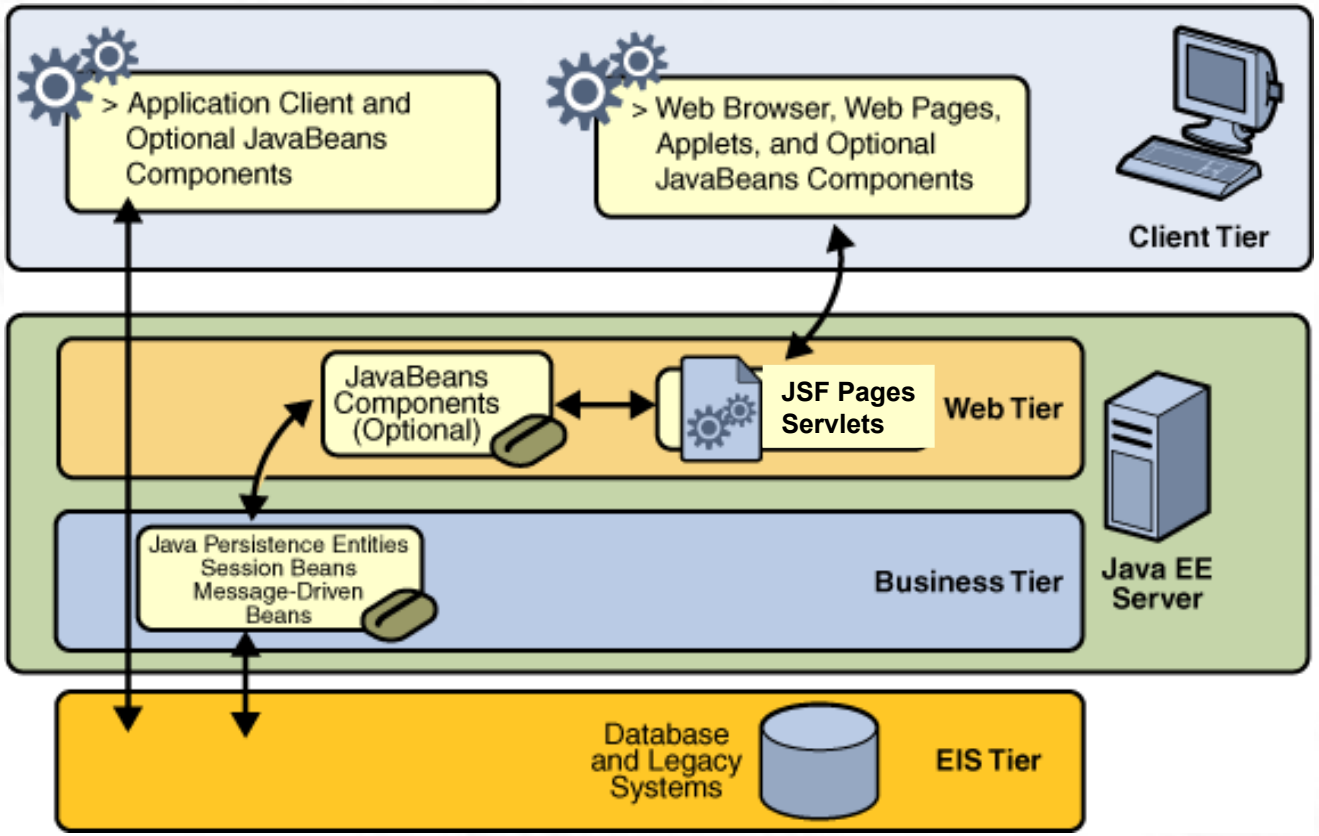
Multi-Tiered Architecture



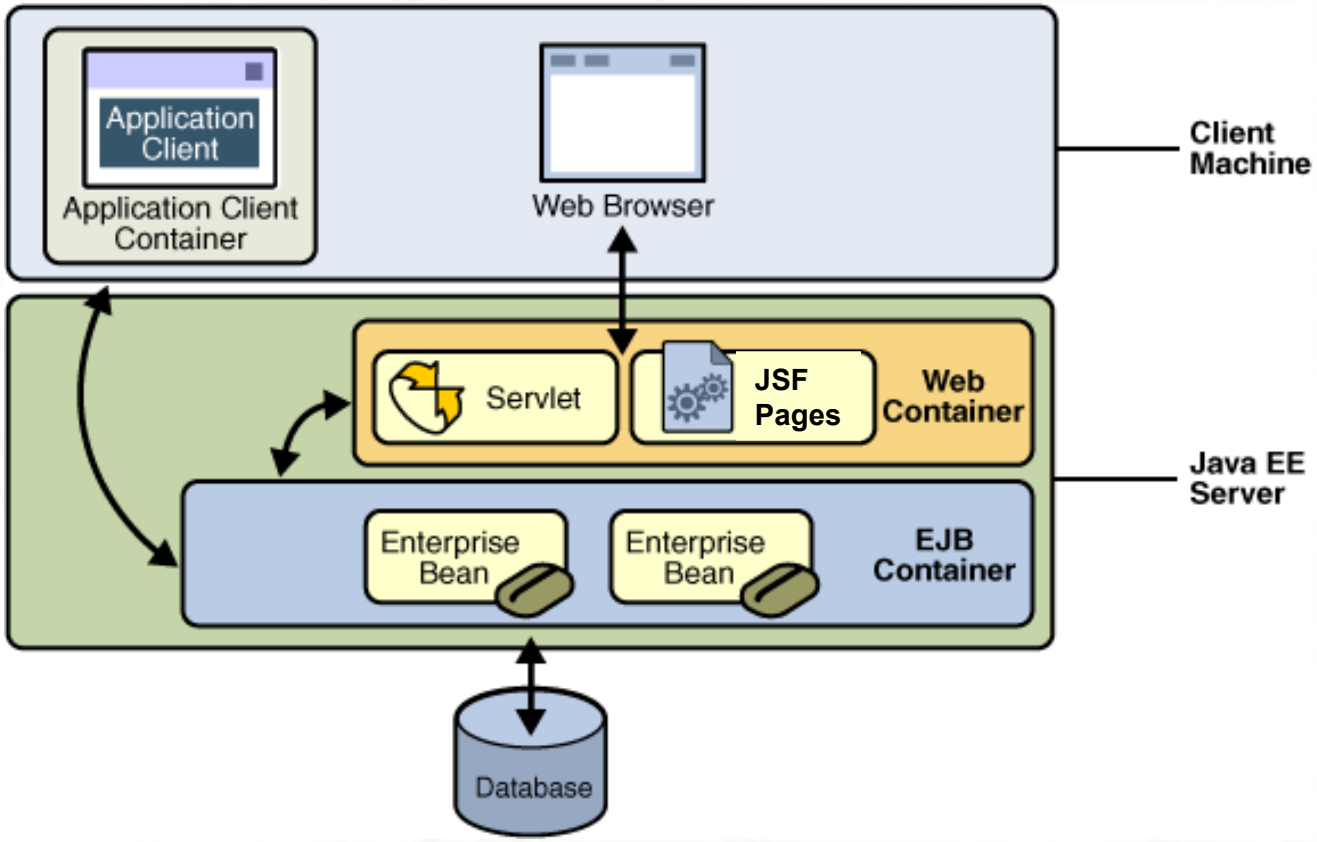
Web Tier and Java EE Applications



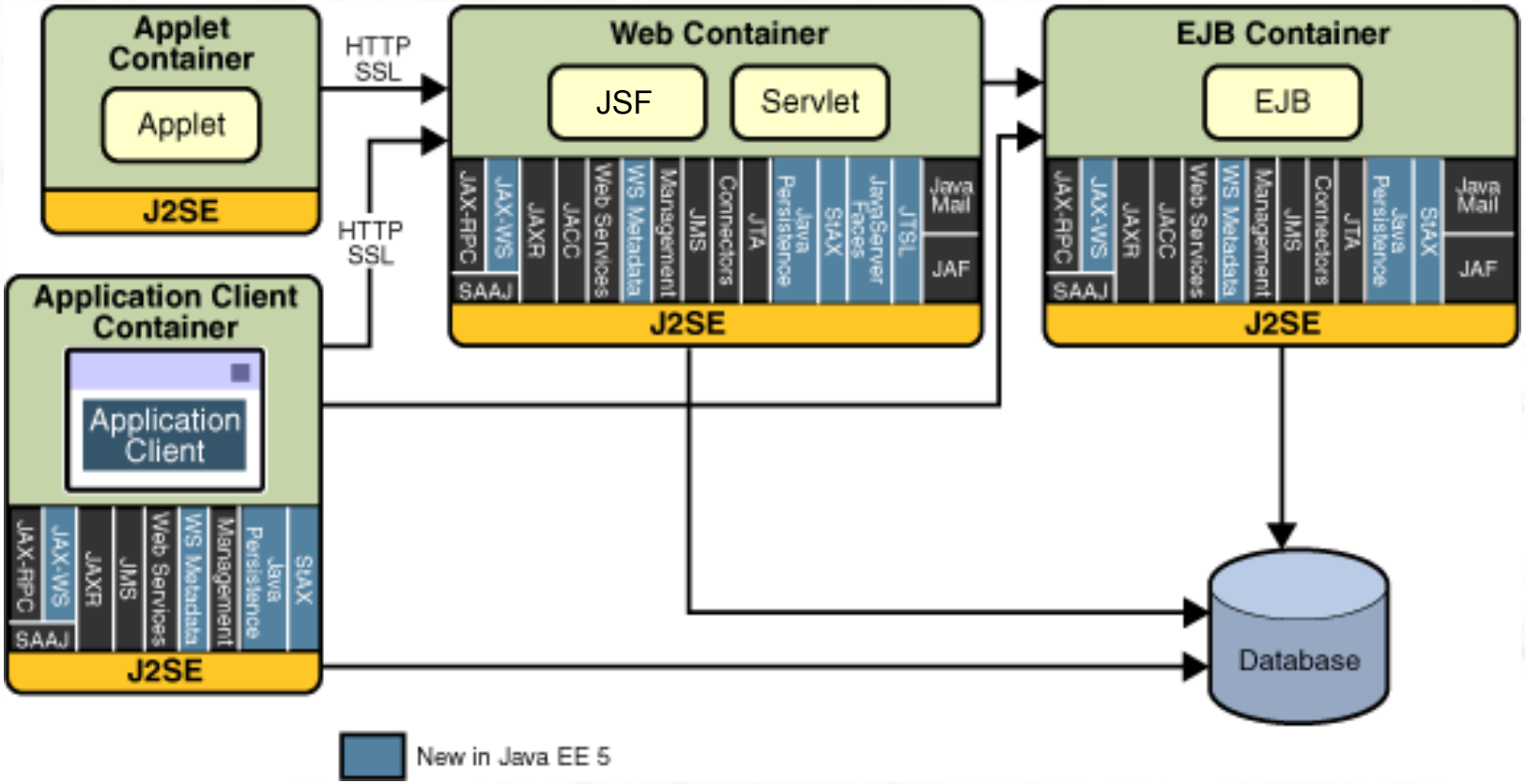
Business and EIS Tiers



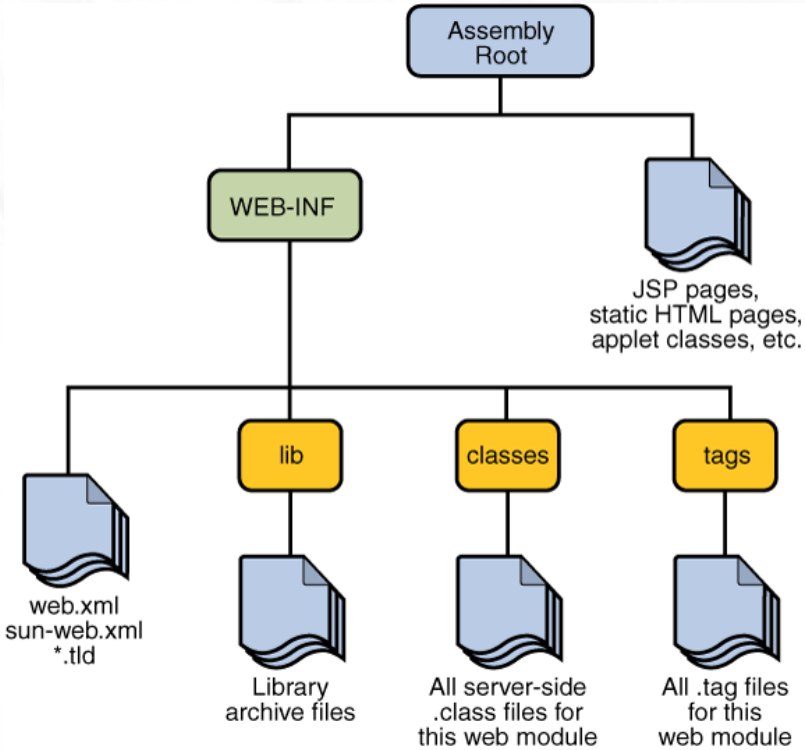
Servers and Containers

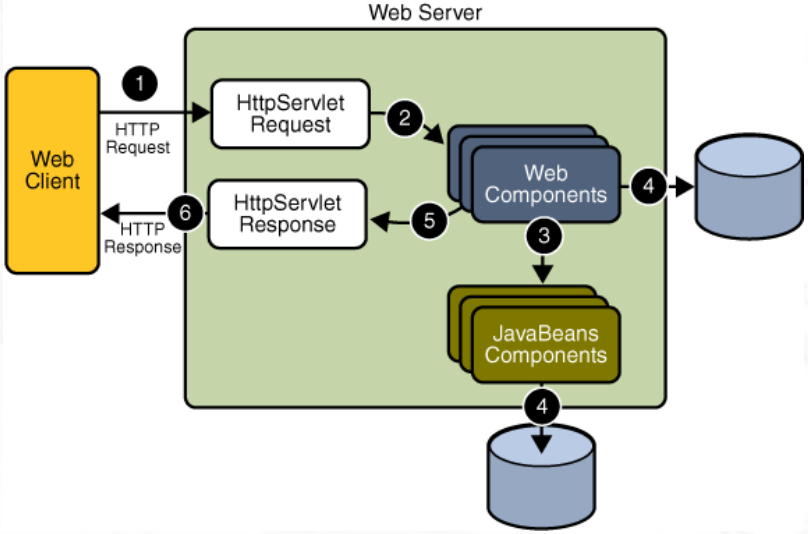


Java EE APIs

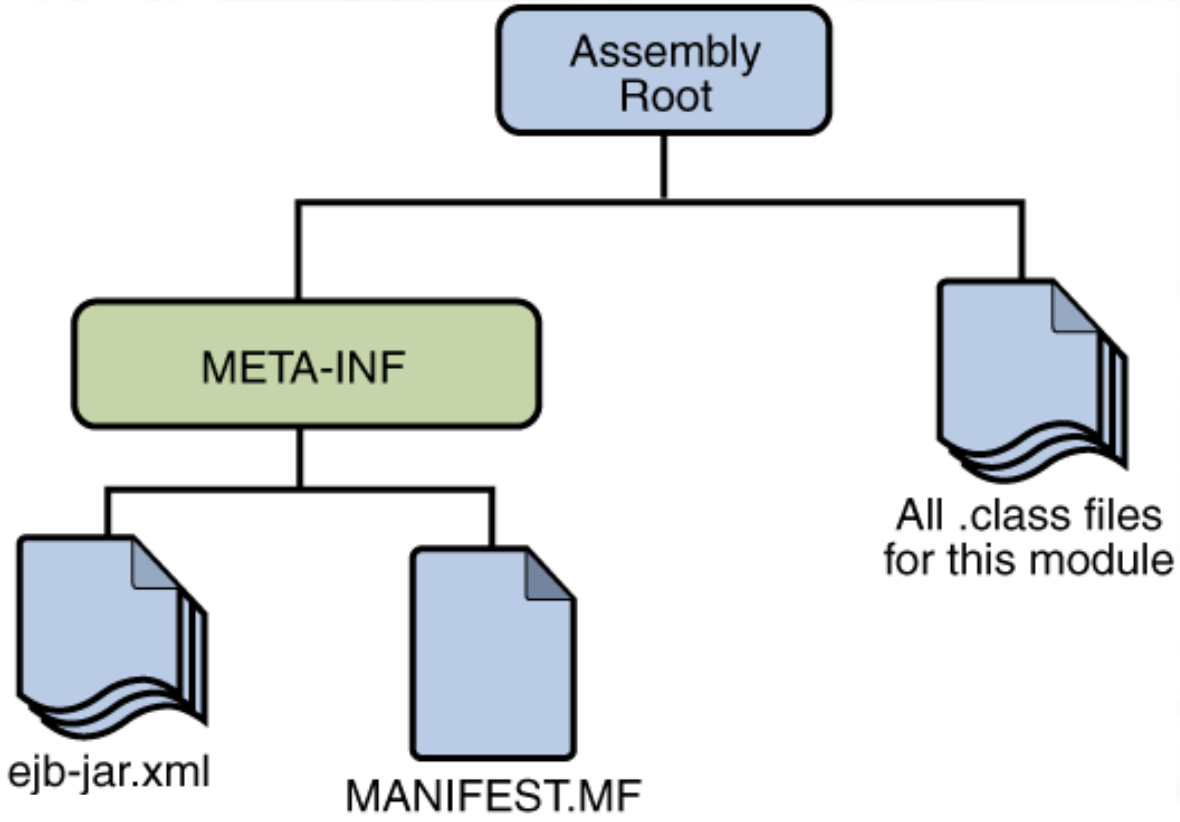

Applet Container
Applet
J2SE
HTTP SSL
HTTP SSL
Web Container
JSF
Servlet
JAX-RPC
JAX-WS
SAAJ
JAXR
JACC
Web Services
WS Metadata
Management
JMS
Connectors
JTA
Java Persistence
StAX
JavaServer Faces
JTSL
Java Mail
JAF
J2SE
EJB Container
EJB
JAX-RPC
JAX-WS
SAAJ
JAXR
JACC
Web Services
WS Metadata
Management
JMS
Connectors
JTA
Java Persistence
StAX
Java Mail
JAF
J2SE
Application Client Container
Application Client
JAX-RPC
JAX-WS
SAAJ
JAXR
JMS
Web Services
WS Metadata
Management
Java Persistence
StAX
J2SE
Database
New in Java EE 5

Web Module Directory Structure - WAR file

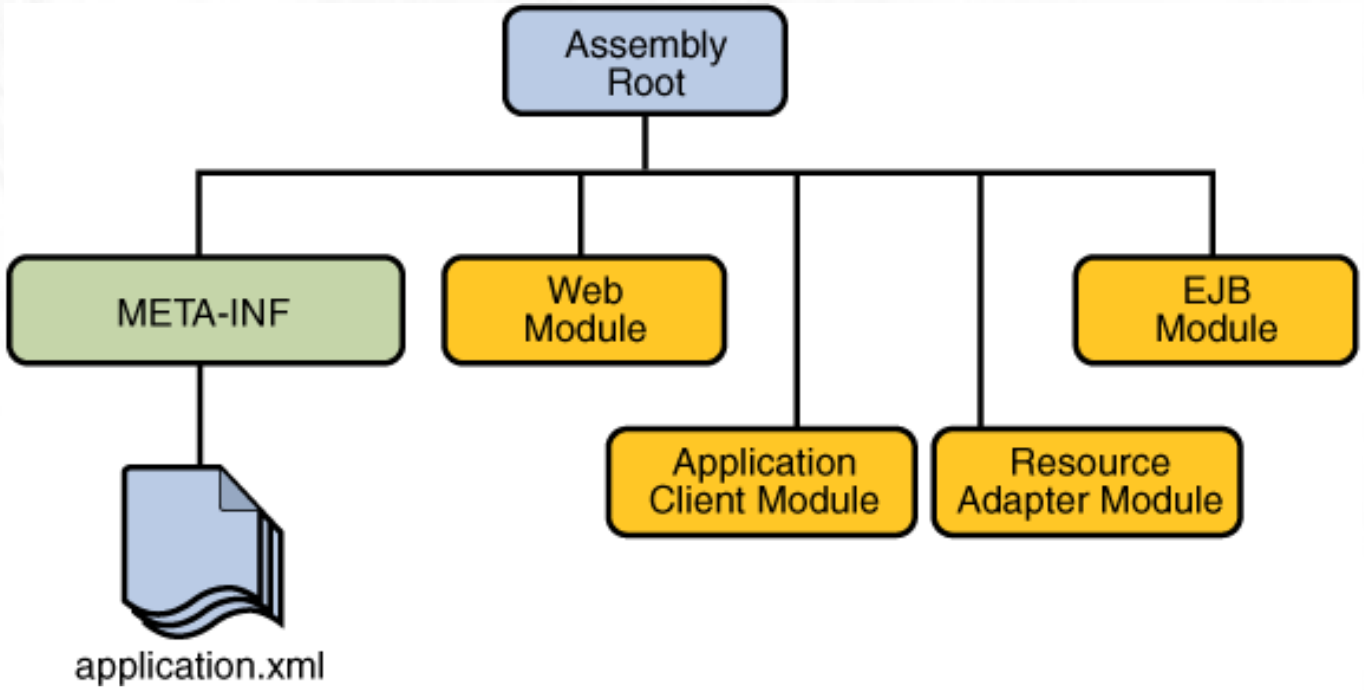




EJB Module - JAR file



Java EE Application Module - EAR file



Java EE Application

- Packaged in zip file with .ear extension
- application.xml deployment descriptor describes the application
- Contains web applications in .war files and ejb modules in .jar files
  - E.g. shipbean.jar and hello.war

application.xml (Optional)

- <application> is root element
  - <display-name> optional display name
  - <description> optional description
  - <module> (one or more) describes each module
    - contains one <ejb>, <web> or <java> element
  - <security-role> (zero or more) describes each global security role
    - contains <description> (optional) and <role-name>

application.xml <module> Element Subelements

- <ejb> contains name of jar file which contains the ejb module
- <java> contains the name of the jar file which contains the java component
- <web> defines a web application
  - <web-uri> defines the .war file which contains the web application
  - <context-root> defines the context root for this web application

Example application.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<application xmlns="http://java.sun.com/xml/ns/j2ee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" version="1.4"
xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
http://java.sun.com/xml/ns/j2ee/application_1_4.xsd">
  <display-name>InfosysApp</display-name>
  <module>
    <web>
      <web-uri>InfosysWeb.war</web-uri>
      <context-root>/infosys</context-root>
    </web>
  </module>
  <module>
    <ejb>infosysBeans.jar</ejb>
  </module>
</application>
```

EJB XML Deployment Descriptors (Optional)

- XML deployment descriptors for EJBs are optional for Java EE 5 onward
  - described throughout the text
- In EJB modules the ejb-jar.xml deployment descriptor goes into the META-INF directory
  - any other XML deployment descriptor files go in the same directory

Structure of ejb-jar.xml File

- <ejb-jar> is root element
  - <enterprise-beans> contains descriptions for each contained ejb
    - <session> or <message-driven> element for each
    - <entity> if using older Entity Bean standard
  - <relationships> contains descriptions for CMR relationships
    - if using older Entity Bean standard
  - <assembly-descriptor> contains security roles, method permissions and transaction attributes

EJB Design

- Java EE 5 introduced a major improvement in light-weight Java Persistence entities
- Best practices evolved as projects used them
  - competes with other object-to-relational persistence technologies such as Hybernate

Design

- Start as usual with requirements
  - use cases OK here
- Go through usual analysis/design process identifying classes
- Structure classes into layers
  - client/web/business/persistence tiers
  - separate web and application clients
    - application clients will access business tier directly

Consider Patterns

- There are several architectural patterns that people use in large systems
- These incorporate experience and what others have found useful
- Not as much information available as there was on J2EE
- See Real World Java EE Patterns

EJB and Entity Bean Identification

- Rules of thumb
  - Entity beans
    - represent records persisted in a database
    - come from nouns in functional description
  - Session beans
    - manage processes or tasks
    - represent activities or taskflow
    - may be façade for entity beans
  - Message-driven beans
    - similar to session beans but driven by asynchronous messages

Identifying Entity Beans

- Find classes in the design that need to be persisted
- These are candidates for entity beans
- Usually significant nouns from functional specification
- Alternatively, any data stored in a database that needs to be accessed in an application

Identifying Session Beans

- Their methods represent things that the business layer of the system can do
- Encapsulate access to data
- Each session bean should represent a collection of related services
  - managing related entity beans
- If a method is almost called in context of another session bean's functions
  - combine the two beans or move method

Identifying Message-Driven Beans

- Message-driven beans encapsulate functionality triggered by an asynchronous message
- Other than messaging, design considerations similar to session beans
- Since MDBs only have onMessage() method, message design is critical
  - message will determine specific function bean is being asked to perform

EJB Issues

- Stateless versus stateful session beans
  - session bean state is not meant to store web or presentation state
  - session bean state is to share information among method calls of same session bean
  - if possible, use stateless session beans
- Local versus remote interfaces
  - remote interfaces less efficient
  - use local interfaces if only called from inside application server

Entity Details

- Returning entity data from EJBs
- Minimizing database access
  - Caching
  - Combining queries
    - use JOIN FETCH rather than lazy loading
- Minimizing updates
  - use FlushModeType COMMIT to delay all updates to end of transaction

Returning Entity Data from EJBs

- Since entity classes are Java objects can use them as data transfer objects
  - making them Serializable
- Be careful of relationship fields that create a complex object graph
- May want a light version with only some of the fields
- Consider using lazy loading and JOIN FETCH queries

Caching

- Often has large impact on performance
- Reduces database load
  - leaves information in memory rather than accessing from database
- Vendor specific
  - see Infinispan Subsystem for Wildfly

Designing Exceptions

- Exceptions are primary error notification mechanism in Java
- In EJBs, exceptions have significant effects on transaction control
- Be aware of which exceptions roll back a transaction and design exceptions accordingly
  - decide for each exception whether it should roll back a transaction

Alternatives to EJBs

- Spring
  - Fierce competition to J2EE 1.4
    - Book: *J2EE without EJBs*, 2004
  - Reaction against heavy-weight entities
  - Based on dependency injection and aspect-oriented programming
    - see http://en.wikipedia.org/wiki/Aspect-oriented_programming
    - transactions, security, logging would be examples of aspects

Alternatives to Java Persistence

- JDBC
  - lower level, more labour intensive
  - can have better performance if do not need security or transactions
- Hibernate
  - similar but with richer API
- iBatis
  - maps SQL result sets into object graphs via XML description
- JDO
  - former alternative to EJB 2.1, now mostly dead

Summary

- Java EE is the latest Java technologies for large, scalable architectures
  - Efficiencies in this version make it suitable for medium size applications as well
- .NET competes on small to medium applications
  - emphasis is more on presentation layer
  - JSF getting comparable
- PHP/Ruby/Perl/Python/etc compete on the solo coder/hacker front
  - Companies wanting low barrier of entry may choose these
  - HTML 5 on the client is strong competitor